

Nama: A. A. Gede Angga Dwipradipta

NIM: 23525024

Mata Kuliah: Deep Learning

Tugas Minggu Ke-10: Transformer untuk Klasifikasi Teks

Tujuan Tugas

Setelah mengerjakan tugas ini, mahasiswa diharapkan mampu: Menjelaskan alur dasar arsitektur Transformer encoder. Mengimplementasikan blok Transformer sederhana untuk klasifikasi teks. Melakukan eksperimen terarah berbasis parameter yang ditentukan dari NIM. Menganalisis hasil training, validasi, dan generalisasi model.

Nilai Parameter Personal Hasil Perhitungan dari NIM

```
{'NIM': '23525024',  
  'LAST4': '5024',  
  'MAXLEN': 150,  
  'EMBED_DIM': 48,  
  'NUM_HEADS': 2,  
  'FF_DIM': 96,  
  'TRAIN_SAMPLES': 4500,  
  'EPOCHS': 3,  
  'DROPOUT': 0.15000000000000002,  
  'BATCH_SIZE': 32,  
  'KODE_VARIAN': 12}
```

Gambar 1. Nilai Parameter Personal Berdasarkan NIM

Ringkasan Teori

Transformer adalah arsitektur Deep Learning yang mengandalkan mekanisme Self-Attention untuk memproses data sekuensial secara paralel, ini mampu menggantikan pendekatan sekuensial tradisional yang digunakan pada RNN atau LSTM (Vaswani et al., 2017). Bagian Encoder pada Transformer bertugas untuk memahami konteks dan representasi matematis dari teks input. Beberapa komponen teoretis utamanya meliputi:

1. **Positional Embedding:** Karena Transformer memproses seluruh kata secara paralel tanpa memedulikan urutan, lapisan ini diperlukan untuk menyuntikkan informasi urutan posisi relatif atau absolut kata ke dalam model (Vaswani et al., 2017). Tanpa fitur ini, model tidak bisa membedakan makna tata bahasa atau posisi kata dalam kalimat.
2. **Multi-Head Attention** adalah mekanisme yang memungkinkan model memusatkan attention pada kata-kata berbeda dalam sebuah kalimat yang saling berkaitan erat. Penggunaan lebih dari satu head memungkinkan model untuk menyeimbangkan penangkapan berbagai representasi fitur secara bersamaan dari sudut pandang yang berbeda (Vaswani et al., 2017).
3. **Feed-Forward Network (FFN)** merupakan jaringan saraf tiruan fully connected di dalam blok Transformer yang diaplikasikan secara identik dan terpisah pada setiap posisi. Lapisan ini memproses keliuaran dari attention layer untuk transformasi fitur tingkat lanjut (Vaswani et al., 2017).
4. **Residual Connection dan Layer Normalization:** Menurut paper He et al. pada tahun 2016, Residual Connection berfungsi meneruskan informasi asli secara langsung melewati sub-layer untuk mencegah masalah vanishing gradient. Sementara itu, Layer normalization (Ba et al.,

2016) menstabilkan distribusi nilai komputasi dari sub-layer sehingga proses pelatihan menjadi jauh lebih stabil dan cepat konvergen.

Arsitektur Model

Model yang dibangun merupakan model klasifikasi biner untuk dataset sentimen IMDB. Arsitektur dalam eksperimen ini berurutan dari input hingga output sebagai berikut:

1. Input Layer: Menerima urutan token sepanjang 150 kata (MAXLEN).
2. Token dan Position Embedding: Mengubah indeks token menjadi vektor *dense* berdimensi 48 (EMBED_DIM) serta menambahkan representasi posisinya.
3. Transformer Block: Lapisan inti yang terdiri dari Multi-Head Attention (NUM_HEADS = 2) dan Feed-Forward Network (FF_DIM = 96), dilengkapi dengan Layer Normalization dan Dropout (0.15).
4. Global Average Pooling 1D: Flatten tensor representasi 3D menjadi 2D dengan mengambil nilai rata-rata tiap dimensi.
5. Dropout (0.15): Dimasukkan untuk mencegah overfitting selama proses pelatihan.
6. Dense Layer (Hidden): Memiliki 20 neuron dengan fungsi aktivasi ReLU.
7. Dropout (0.15): Lapisan regularisasi tambahan.
8. Output Layer (Dense): 1 neuron dengan fungsi aktivasi Sigmoid untuk menghasilkan probabilitas klasifikasi biner.

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 150)	0
token_and_position_embedding (TokenAndPositionEmbedding)	(None, 150, 48)	967,200
transformer_block (TransformerBlock)	(None, 150, 48)	28,320
global_average_pooling1d (GlobalAveragePooling1D)	(None, 48)	0
dropout_3 (Dropout)	(None, 48)	0
dense_2 (Dense)	(None, 20)	980
dropout_4 (Dropout)	(None, 20)	0
dense_3 (Dense)	(None, 1)	21

Total params: 996,521 (3.80 MB)
 Trainable params: 996,521 (3.80 MB)
 Non-trainable params: 0 (0.00 B)

Gambar 2. Arsitektur Model dan Total Parameter

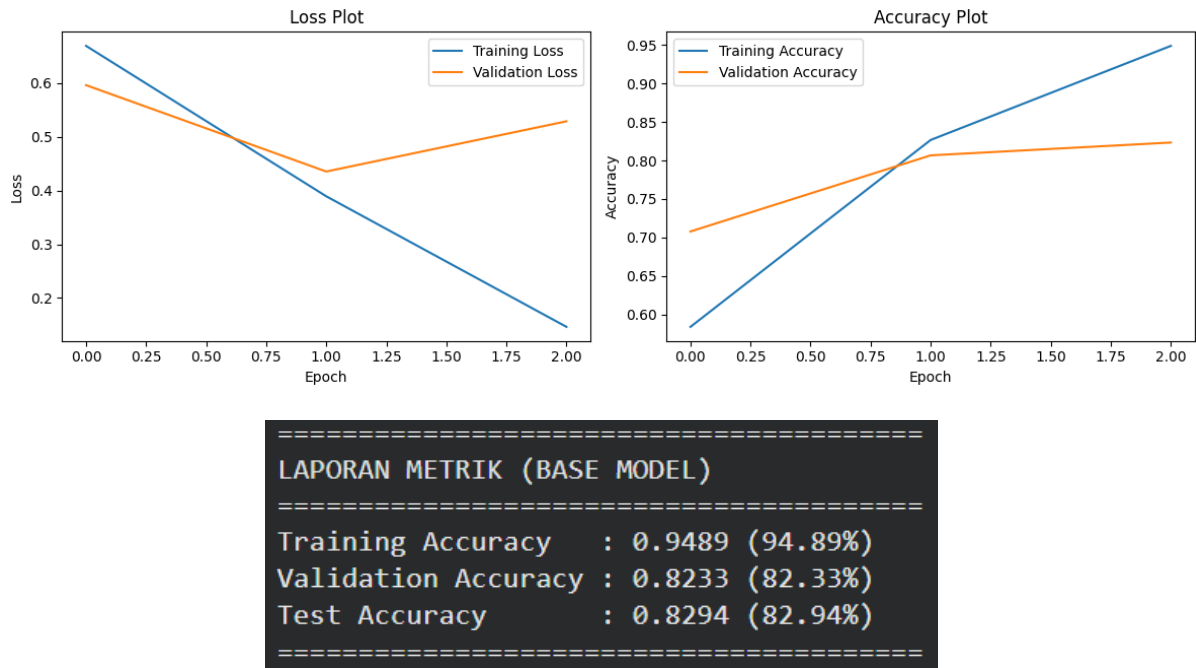
Setup Eksperimen

Model dilatih dengan dataset sentimen film IMDB dengan ukuran kosakata dibatasi 20.000 kata. Data training dipotong sesuai parameter konfigurasi, lalu displit dengan rasio 80% untuk training murni dan 20% untuk validation. Model dikompilasi menggunakan optimizer Adam dan loss function binary crossentropy. Untuk menganalisis pengaruh parameter, tiga eksperimen variasi dilakukan dengan Base Model sebagai pembanding:

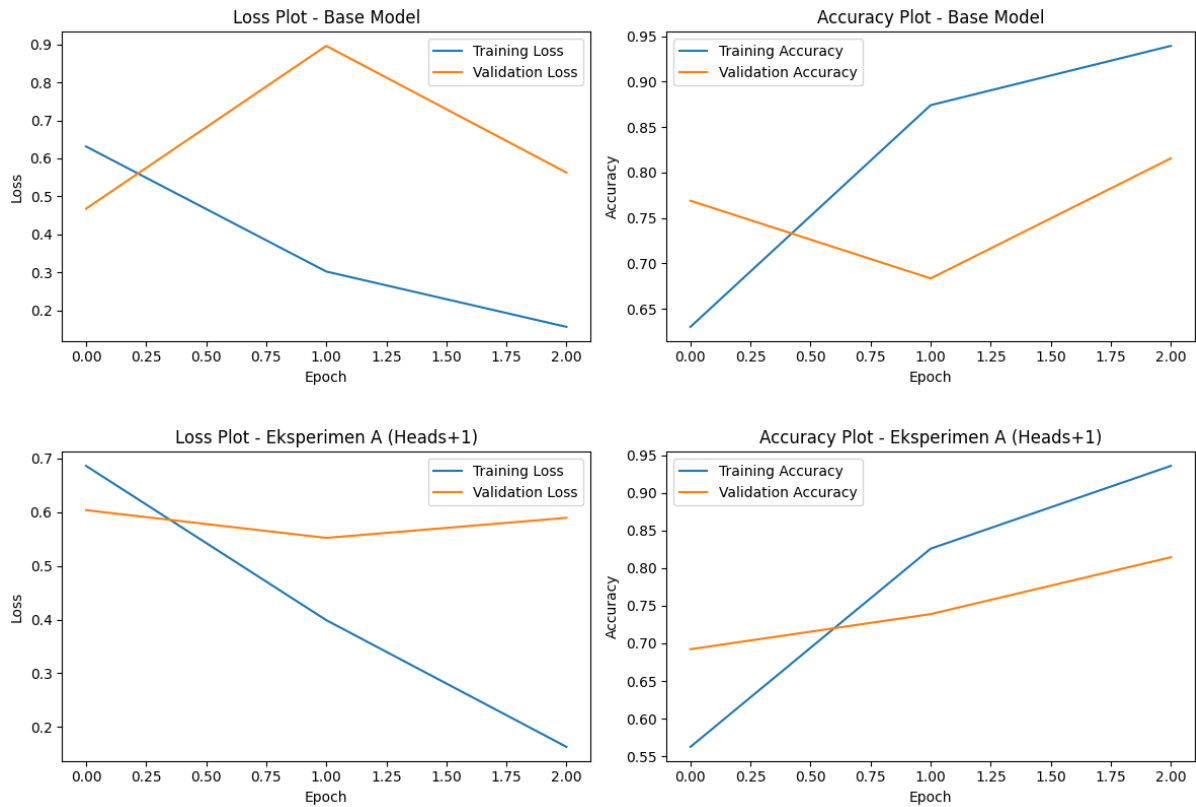
1. Eksperimen A: Mengubah NUM_HEADS dari 2 menjadi 3.
2. Eksperimen B: Menggandakan FF_DIM dari 96 menjadi 192.

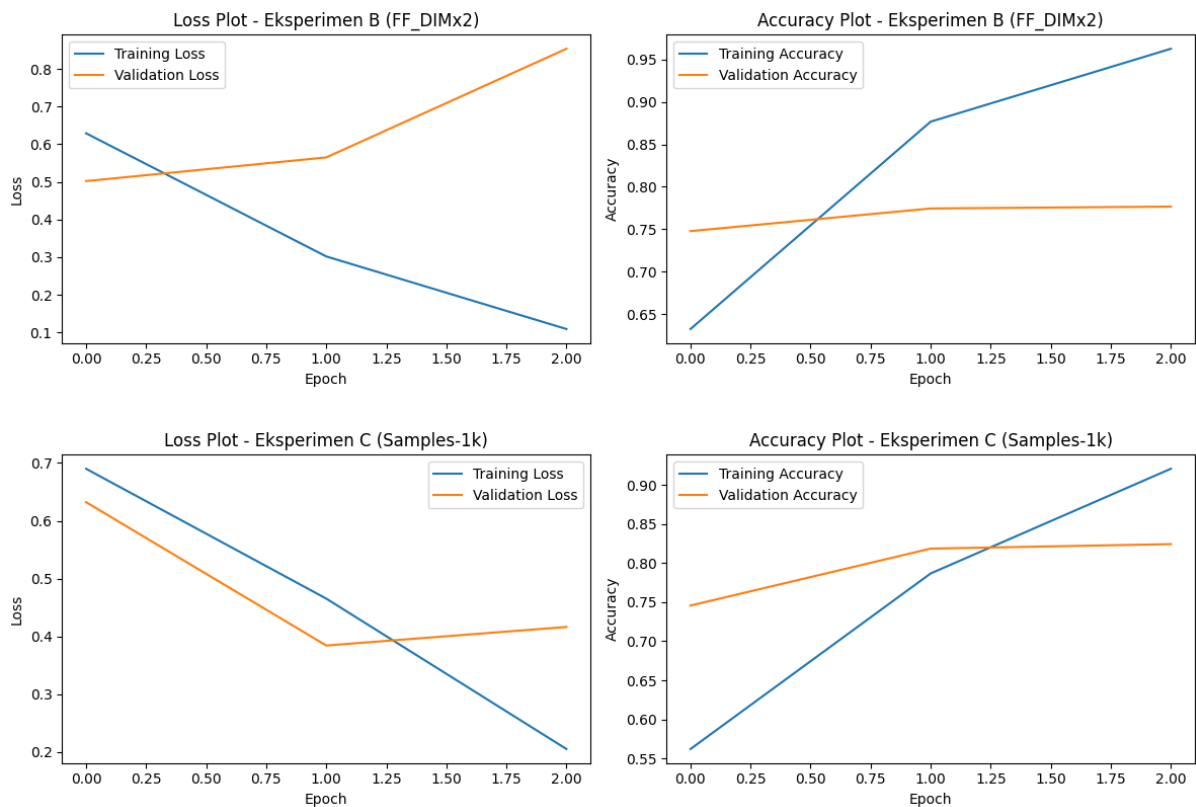
3. Eksperimen C: Mengurangi TRAIN_SAMPLES sebesar 1.000 menjadi 3.500 sampel.

Hasil dan Tabel Perbandingan



Gambar 3. Grafik Plot Accuracy dan Loss beserta Laporan Metrik Base Model





Gambar 4. Perbandingan Grafik Plot Loss dan Accuracy Semua Eksperimen

Seluruh model dilatih selama 3 epoch. Evaluasi akhir diuji menggunakan keseluruhan data test bawaan dari dataset IMDB. Hasil dapat dilihat pada tabel dibawah ini:

Eksperimen	NUM_HEADS	FF_DIM	TRAIN_SAMPLES	Train Acc	Val Acc	Test Acc
Base Model	2	96	4500	93.94%	81.56%	83.19%
Eksperimen A (Heads+1)	3	96	4500	93.58%	81.44%	82.52%
Eksperimen B (FF_DIMx2)	2	192	4500	96.25%	77.67%	78.70%
Eksperimen C (Samples-1k)	2	96	3500	92.07%	82.43%	81.92%

#Catatan: Jika dilihat lebih detail pada **Gambar 3** dan **Gambar 4** (khususnya Plot Hasil Base Model) Terdapat sedikit perbedaan nilai antara evaluasi awal (Laporan Metrik Base Model) dan evaluasi pada tabel eksperimen yang disebabkan oleh variasi stokastik pada inisialisasi bobot dan pengocokan data. Namun, setiap eksperimen dijalankan dengan inisialisasi model baru untuk memastikan tidak ada kebocoran informasi antar eksperimen. Ini dibuktikan dengan code “model = tf.keras.Model(inputs=inputs, outputs=outputs)” di dalam fungsi **run_experiment**. Setiap kali fungsi ini dipanggil untuk "Base Model" maupun "Eksperimen A/B/C", program akan menghapus model lama dari memori dan membangun model yang benar-benar baru dengan bobot yang masih kosong dan acak.

Analisis

Berdasarkan metrik Test Accuracy pada eksperimen Google Colab, didapatkan hasil bahwa Base Model dengan konfigurasi NUM_HEADS = 2, FF_DIM = 96, dan TRAIN_SAMPLES = 4500 adalah konfigurasi terbaik dengan akurasi 83.19%. Jika dilihat lagi, tingkat overfitting-nya memang yang paling moderat dibandingkan varian eksperimen lainnya. Mengapa hasil tiap eksperimen berbeda dijelaskan sebagai berikut:

- Eksperimen A (Penambahan Attention Head): Penambahan attention head bertujuan agar model menangkap lebih banyak fitur paralel. Namun, Test Acc sedikit menurun menjadi 82.52%. Peningkatan kompleksitas attention ini tidak diimbangi dengan jumlah data yang masif, sehingga model rentan menangkap noise ketimbang pola semantik yang umum.
- Eksperimen B (Penggandaan FF_DIM): Model ini mencetak Training Accuracy tertinggi yaitu 96.25% dibanding model lain, tetapi Test Accuracy-nya anjlok ke 78.70%. Fenomena ini adalah bukti model mengalami overfitting akibat kapasitas memori model yang terlalu besar. Parameter yang membengkak membuat model hanya menghafal data latih dan gagal menggeneralisasi pengetahuan untuk data uji.
- Eksperimen C (Pengurangan Data Latih): Akurasi tes turun menjadi 81.92%. Pemangkasan 1.000 sampel ini menyebabkan model kehilangan banyak variasi kosakata. Meski Training Accuracy tetap tinggi, hilangnya variasi data menyebabkan kemampuan generalisasi memburuk.

Pelatihan dibatasi hanya 3 epoch sesuai spesifikasi NIM dan dengan subset data yang sangat kecil untuk skala Transformer. Ada kemungkinan model belum mencapai konvergensi penuh dan rentan terhadap instabilitas batch.

Kesimpulan

Eksperimen ini membuktikan secara empiris bahwa memperbesar kapasitas model Transformer baik itu dengan menambah jumlah Head maupun memperlebar dimensi jaringan Feed-Forward tidak selalu berbanding lurus dengan peningkatan performa. Pada dataset yang berukuran relatif kecil pada eksperimen ini dengan range 3.500 – 4.500 sampel, arsitektur model yang terlalu besar justru memicu masalah overfitting karena model cenderung menghafal data latih. Kunci utama performa Transformer terletak pada ketersediaan dan variasi jumlah data pelatihan atau training samples yang masif.

Daftar Pustaka

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems* (NeurIPS), 30.
2. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer Normalization. *arXiv preprint arXiv:1607.06450*.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

Link Akses Google Colab:

<https://colab.research.google.com/drive/1IESjTdxgmTwQe14fXQI617NLv1f4-dSH?usp=sharing>